System Architecture Overview

1. Architecture Summary

The SpendWise expense tracker is built using a Modular Monolith architecture. In simple terms, this means the entire application is built as a single, cohesive unit, but it is organized internally into distinct, independent sections. This approach is ideal for a focused application like SpendWise because it balances high performance with ease of maintenance.

By choosing a modular monolith, we ensure that the system is straightforward to develop and deploy while remaining organized enough to grow. If a specific part of the app—such as the reporting engine—needs to become more complex in the future, its modular nature allows us to upgrade it without needing to rewrite the entire system. This architecture provides a solid foundation that is both cost-effective for current needs and flexible for future growth.

2. Key System Components

The system is divided into three primary layers, each with a specific responsibility:

- The User Interface (Frontend): This is the "face" of the application that users interact with in their web browsers. Built using modern web technologies (React), it is responsible for presenting data clearly, capturing user input through forms, and displaying interactive financial charts. It focuses entirely on the user experience, ensuring the app is responsive and easy to navigate.
- The Application Brain (Backend API): This is the central hub where all the "thinking" happens. When a user clicks a button on the interface, the backend receives the request, verifies who the user is, performs the necessary calculations, and communicates with the database. It acts as a gatekeeper, ensuring that only authorized users can access or change their financial data.
- The Data Vault (Database): This is where all information is permanently stored. We use a professional-grade relational database (PostgreSQL) to ensure that every expense, user profile, and category is saved securely and accurately. It is designed to handle complex relationships, such as linking thousands of individual expenses to specific user accounts and categories.

3. Data Flow

To understand how the system works in practice, here is the step-by-step journey of a typical action, such as adding a new expense:

1. User Input: The user enters the expense details (amount, category, and date) into a form on the web interface and clicks "Save."
2. Secure Transmission: The web interface packages this information and sends it securely to the Backend API, along with a "digital ID badge" (token) that proves the user is logged in.
3. Validation & Security: The Backend API receives the data, checks that the "ID badge" is valid, and ensures the expense details are formatted correctly (e.g., the amount is a positive number).
4. Storage: Once validated, the Backend API instructs the Database to save the new record. The database confirms the save was successful.
5. Confirmation: The Backend API sends a success message back to the web interface.
6. Visual Update: The web interface immediately updates the user's dashboard and charts to reflect the new expense, providing instant visual feedback.

4. Database Design

The database is the foundation of the system’s reliability. We use a Relational Database Management System, which is the industry standard for financial data because it excels at maintaining "data integrity"—ensuring that numbers always add up and records never go missing.

The data is organized into three main areas:

- User Profiles: Stores login credentials (encrypted for safety) and account settings.
- Expense Records: The core data of the app, including amounts, descriptions, and timestamps. Each record is strictly tied to a specific user.
- Categories: A library of labels (like "Rent," "Groceries," or "Entertainment") that help users organize their spending.

By separating data this way, the system can quickly generate reports, such as "How much did I spend on Groceries last month?" by instantly filtering and totaling the relevant records.

5. External Integrations

In its current version, SpendWise is designed to be a self-contained system to maximize privacy and speed.

- Internal Analytics: Rather than sending your financial data to a third-party service, the system performs all spending calculations and report generation internally.
- Extensibility: While no external APIs are required for the core launch, the architecture is "integration-ready." This means that in future versions, we could easily add connections to services like bank feeds, currency conversion APIs, or automated receipt scanning tools without restructuring the core system.

6. Scalability & Performance

Even though SpendWise starts as a simple tool, it is engineered to handle a growing number of users and data points:

- Efficient Processing: The backend uses "asynchronous" technology, which allows it to handle many user requests simultaneously without slowing down.
- Database Optimization: We use "indexing" on the database, which acts like an index in a book, allowing the system to find a specific user's expenses instantly, even if there are millions of records in the system.
- Resource Management: Because the system is modular, we can scale it "vertically" by giving the server more power, or "horizontally" by running multiple copies of the backend to distribute the workload as the user base grows.

7. Security Considerations

Security is our highest priority, especially when dealing with personal financial information. We implement a multi-layered security strategy:

- Identity Protection: We use JSON Web Tokens (JWT). When a user logs in, they receive a secure digital key. This key is required for every subsequent action, ensuring that no one can "peek" at another user's data.
- Data Privacy: The system is built with strict "data isolation." Every time the database is asked for information, the system automatically adds a filter to ensure it only retrieves data belonging to the currently logged-in user.
- Password Safety: We never store actual passwords. Instead, we use advanced

mathematical "hashing" (Passlib). Even if someone gained access to the database, they would only see scrambled code that cannot be turned back into a password.

- Secure Communication: All data traveling between the user’s browser and our servers is encrypted, preventing "man-in-the-middle" attacks.

8. Deployment Overview

To ensure the application runs perfectly regardless of the server environment, we use Docker.

Think of Docker as a "shipping container" for software. We package the frontend, the backend, and the database into their own containers. These containers include everything the software needs to run—code, tools, and settings.

We use Docker Compose to manage these containers. With a single command, the entire SpendWise ecosystem (the web interface, the brain, and the vault) starts up and connects automatically. This makes the deployment process highly predictable, reduces "it works on my machine" errors, and makes it easy to move the app between different cloud providers in the future.

9. Future Improvements

The current architecture is a "Version 1.0" designed for stability. However, we have paved the way for several exciting future enhancements:

- Mobile Application: The Backend API is already designed to support a mobile app (iOS/Android) in addition to the web interface.
- Automated Insights: We can add a "Notification Module" to alert users when they are nearing their budget limits for a specific category.
- Receipt Scanning: The system could be expanded to allow users to upload photos of receipts, using AI to automatically fill out the expense form.
- Data Export: Adding the ability to export financial reports into Excel or PDF formats for tax purposes or personal archiving.